



**Tecnológico
de Monterrey**

Evidencia 1. Resaltador de sintaxis

Daniel Armas A01786896

Emilio Torres A01787141

Guillermo González A01787393

Se eligió C++ por ser un lenguaje ampliamente conocido por nosotros, además de que tiene una estructura léxica clara y bien definida. Las categorías léxicas identificadas son las siguientes:

Token	Ejemplos	Color en el resaltado
Palabra reservada	int, float, if, else, for, while, return, class, void, const, char, string, using, namespace, true, false	Azul
Identificador	main, edad, contador, saludo	Negro
Entero	20, 0, 25	Naranja oscuro
Real	3.14, 2.50, 0.001	Naranja
Carácter	'a', 'Z', '9'	Verde claro
Cadena (string)	"Hola", "Guillermo"	Verde oscuro
Operador	+, -, *, =, <, >, !, &&, , %, ^	Rojo
Delimitador	;, ,, (,), [,], {, }	Gris
Comentario	// texto	Gris
Directiva / librería	#include <iostream>	Morado

El autómata finito determinístico se construyó con 14 estados y 11 columnas de entrada, representadas en la tabla de transición implementada directamente en Python. Cada columna corresponde a una categoría de carácter:

Columna	Categoría	Caracteres que la activan
0	letra	a-z, A-Z, _
1	dígito	0-9
2	punto	.
3	comilla simple	'
4	comilla doble	"

5	operador	+, -, =, *, <, >, !, &, , %, ^
6	delimitador	;, ,, (,), [,], {, }
7	slash	/
8	gato	#
9	espacio	espacio, \n, \t
10	otro	cualquier otro carácter

Los estados y su significado son:

Estado	Descripción	Token que acepta
0	Inicial	
1	Leyendo identificador o palabra reservada	Identificador / Keyword
2	Leyendo entero	Entero
3	Leyendo punto después de entero	(transición a real)
4	Leyendo parte decimal	Real
5	Inicio de carácter (tras ')	(transición)
6	Contenido del carácter	(transición)
7	Fin del carácter (segunda ')	Char
8	Leyendo string (tras ")	(transición)
9	Operador aceptado	Operador
10	Delimitador aceptado	Delimitador
11	Primer / leído	(posible comentario)
12	Leyendo comentario //	Comentario
13	Leyendo directiva #include	Directiva / Librería
14	Fin de string (segunda ")	String

La tabla de transición completa implementada es:

Estado	letra	dig	punto	c_s	c_d	op	delim	/	#	esp	otro
0	1	2	-1	5	8	9	10	11	13	0	-1
1	1	1	-1	-1	-1	-1	-1	-1	-1	-1	-1
2	-1	2	3	-1	-1	-1	-1	-1	-1	-1	-1
3	-1	4	-1	-1	-1	-1	-1	-1	-1	-1	-1
4	-1	4	-1	-1	-1	-1	-1	-1	-1	-1	-1
5	6	6	6	7	6	6	6	6	6	6	6
6	-1	-1	-1	7	-1	-1	-1	-1	-1	-1	-1
7	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
8	8	8	8	8	14	8	8	8	8	8	8
9	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
10	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
11	-1	-1	-1	-1	-1	-1	-1	12	-1	-1	-1
12	12	12	12	12	12	12	12	12	12	12	12
13	13	13	13	13	13	13	13	13	13	-1	13
14	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

Un valor de -1 indica que no existe transición válida desde ese estado con ese carácter, lo que señala el fin del token actual y provoca que el lexer emita el token acumulado y regrese al estado 0.

El analizador léxico funciona leyendo el archivo carácter por carácter y usando una tabla para decidir qué hacer en cada paso. Esto hace que sea muy eficiente, porque el tiempo que tarda depende directamente del tamaño del archivo $O(n)$, es decir, procesa cada carácter solo una vez. Gracias a esto, funciona bien incluso con archivos grandes, aunque tiene algunas limitaciones, como no manejar errores de forma muy clara. Aun así, estas mejoras se pueden agregar sin cambiar la lógica principal. En cuanto a memoria, guarda todo el archivo en una sola variable, lo que podría llegar a ser un problema con archivos muy grandes.

Por otro lado, este tipo de tecnología es la base de herramientas que usamos todos los días, como compiladores y editores de código, y ayuda a entender mejor cómo funcionan internamente. Sin embargo, también puede tener usos negativos, como analizar código sin permiso, detectar vulnerabilidades automáticamente o incluso monitorear el trabajo de programadores sin que lo sepan. Por eso, aunque es una herramienta muy útil, es importante usarla con responsabilidad y ser consciente de sus posibles implicaciones.

DFA

